

오늘의 학습내용

✓ 17 스트림 요소 처리

○ 중간 처리

- `distinct()`: 중복제거
- `limit(n)`: 지정한 개수로 제한
- `filter(Predicate)`: 지정한 조건을 만족한 요소로만 새로운 스트림 생성
- `map(Function)`: 요소의 타입 변환
- `flatMap(Function)`: 2차원 스트림을 1차원 스트림으로 평탄화
- `sorted()`, `sorted(Comparator)`

○ 최종처리

- `forEach(Consumer)`
- `allMatch(Predicate)`, `anyMatch(Predicate)`, `noneMatch(Predicate)`
- `count()`, `sum()`, `average()`, `min()`, `max()`, `findFirst()`
- `toList()`, `collect(Collectors.toSet())`,

○ Optional

- 리턴값이 없을 수 있는 상황에서 사용
- `orElse(기본값)`: 값이 있으면 값을 리턴, 값이 없으면 기본값 리턴
- `orElseThrow(Supplier)` : 해당 예외 객체로 예외를 발생
 - `orElseThrow(NoSuchElementException::new)`
- `ifPresent(Consumer)` : 값을 매개변수로 전달받아 처리

✓ 18 데이터 입출력

바이트 스트림		문자 스트림	
입력 스트림	출력 스트림	입력 스트림	출력 스트림
<code>InputStream</code>	<code>OutputStream</code>	<code>Reader</code>	<code>Writer</code>
<code>XXXInputStream</code> (<code>FileInputStream</code>)	<code>XXXOutputStream</code> (<code>FileOutputStream</code>)	<code>XXXReader</code> (<code>FileReader</code>)	<code>XXXWriter</code> (<code>FileWriter</code>)

○ OutputStream

리턴 타입	메소드
void	<code>wrtie(int b)</code>
void	<code>write(byte[] b)</code>
void	<code>write(byte[] b, int off, int len)</code>
void	<code>flush()</code>
void	<code>close()</code>

○ InputStream

리턴 타입	메소드
int	<code>read()</code>
int	<code>read(byte[] b)</code>
void	<code>close()</code>